BA ZC581 :
Applied Data Science
Module: 12

BITS Pilani

Pilani | Dubai | Goa | Hyderabad

J A Gokhale.

Disclaimer

- The slides presented here are obtained / extracted from the authors of the prescribed books and from various other subject contributors.
- I hereby thank and acknowledge all the contributors for their material and inputs.
- I have added and modified a few slides to suit the requirements of the course.

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Module 12:
Data Warehousing and Pipelines Basics

Module 10 Topics

Topic 1: Data pipelines

Topic 2: Introduction to ETL / ELT

Topic 3: Data Lakes & Data Warehousing

Topic 4: Batch jobs

Topic 5: Airflow / Cloud Overview

Ref: T1 , Ch 4

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Topic 1: Data Pipelines

A typical Scenario

- Organizations want an efficient flow of data from one location to another (a SaaS application to a data warehouse)
  - This is one of the most critical operations in today's data-driven enterprise.
  - Useful analysis cannot begin until the data becomes available.
- Enterprise data flows can be precarious
  - things can go wrong during data transportation from one system to another
  - data can get corrupted,
  - it can hit bottlenecks (causing latency),
  - data sources may conflict and/or generate duplicates.
- As the complexity of the requirements grows and the number of data sources multiplies, these problems increase in scale and impact the performance of the system.
- **Solution: Data pipelines**

Data Pipeline..

- Generally speaking, the data pipeline defines how data flows with the enterprise for various data needs.
- It is the **end-to-end process / system that moves data from one place to another, transforms it along the way, and makes it ready for use -** for analytics, dashboards, machine learning, or business applications

Data Pipeline

- A **data pipeline** is a structured sequence of steps that:
  - **Collects data** from one or more sources
  - **Moves it** through a series of processing stages
  - **Transforms it** into a usable format
  - **Delivers it** to a destination such as a database, warehouse, lake, or BI tool
- It is like a **plumbing system** of the data world—automated, reliable, and repeatable
- Also handles many other aspects like security, serialization, transfer protocol management and others

Data pipeline

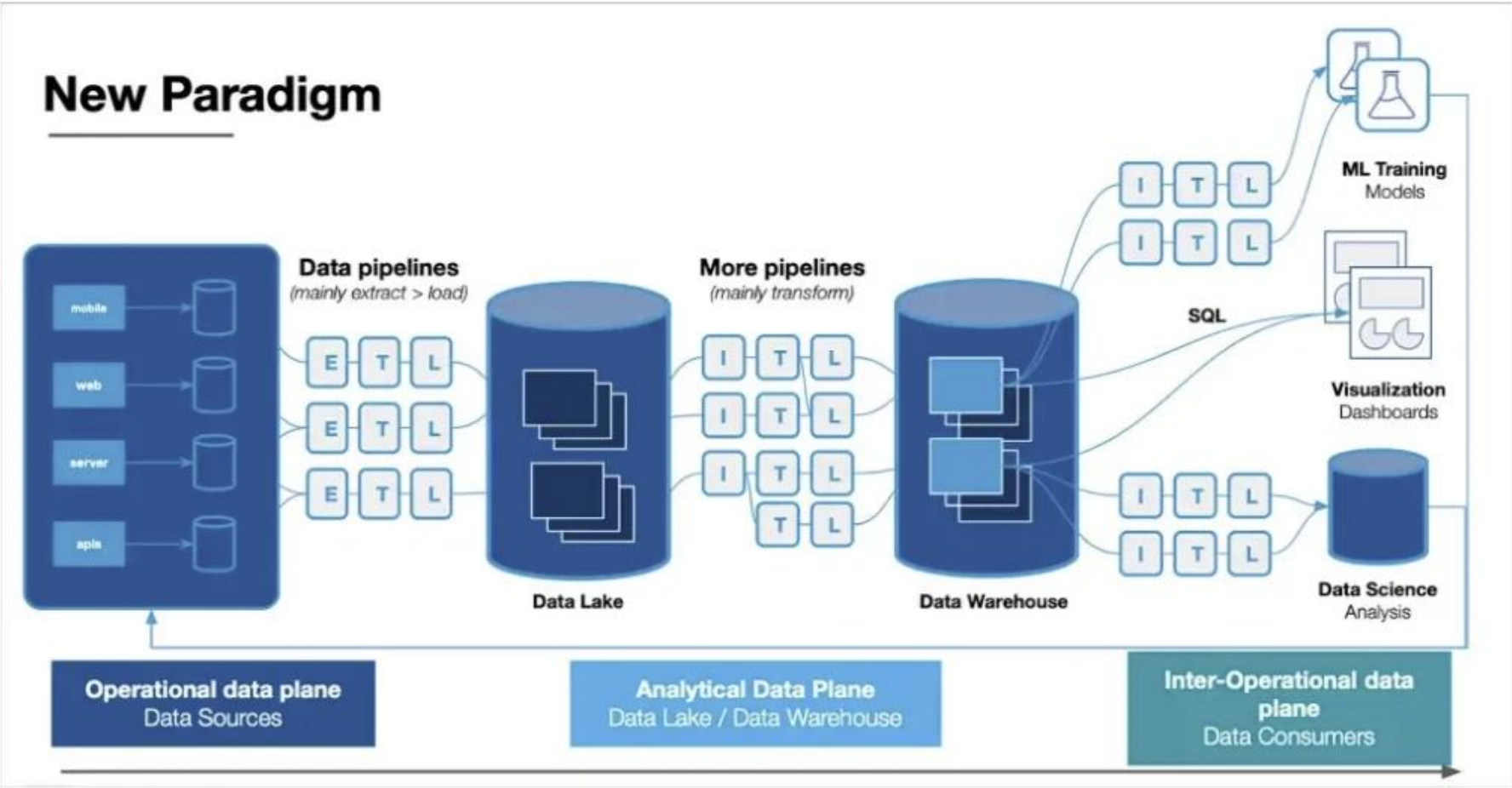
- views all data as streaming data
- it allows for flexible schemas
- handles data from static sources (like a flat-file database) or real-time sources (such as online retail transactions, media streams)
- divides each data stream into smaller chunks and feeds in parallel to various consumers
  - This data is fed to processing engines to perform preprocessing engines for
    - Transformations
    - Filtering
    - Sorting
    - Aggregation

    The ETL pipeline!
  - Finally stored into warehouses

The ETL process

- Key component of the data pipeline
- Extract : gathering data from heterogeneous sources
- Transform : preprocess data to make them suitable for analysis
- Load : Store the data into Data Warehouse / Data Lakes etc

A typical Data pipeline Architecture



Example

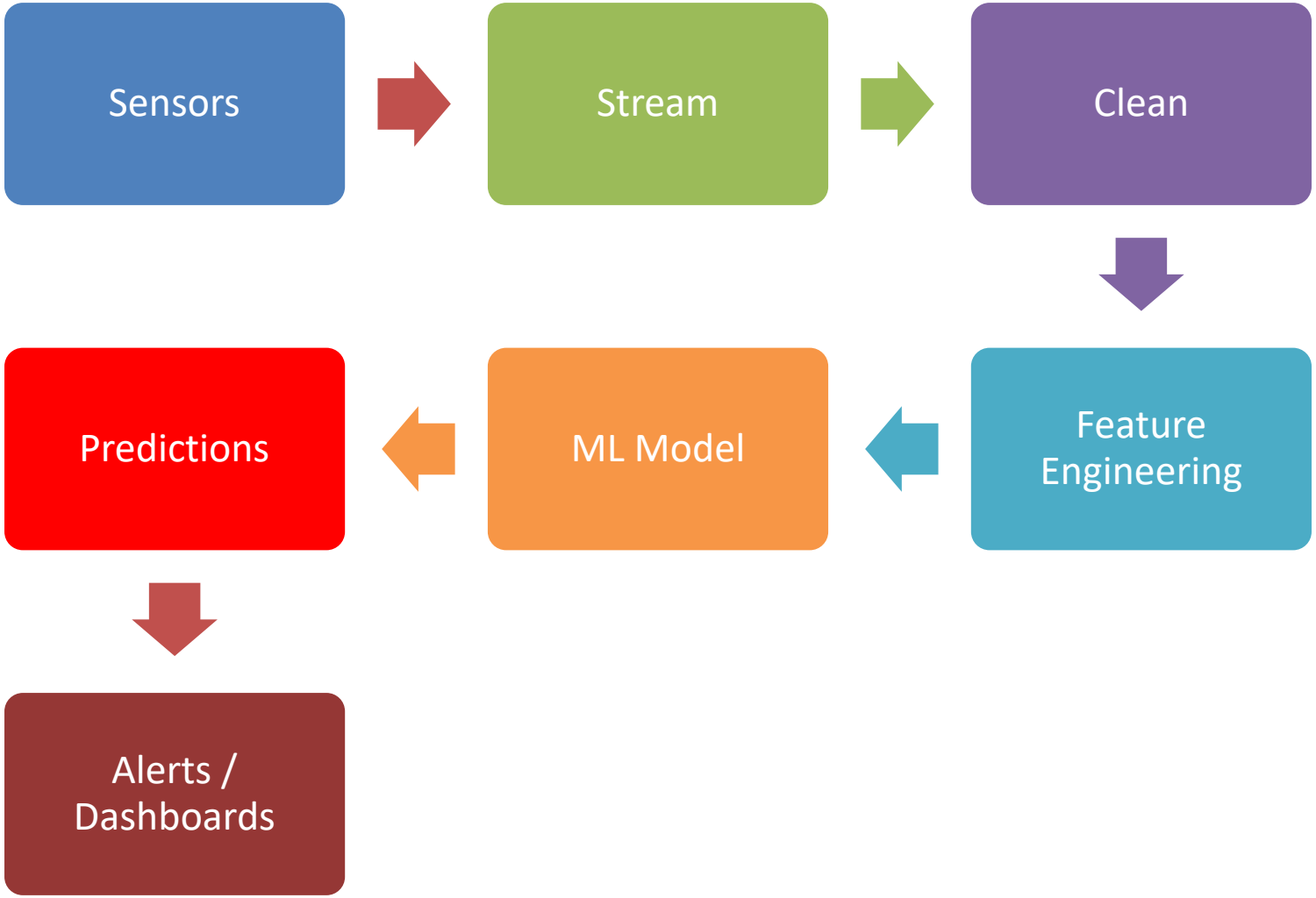
Scenario: Retail company wants daily sales dashboards.

Pipeline:

- Extract sales data from POS →
- Clean & aggregate by store →
- Load into a warehouse and append data →
- Refresh Power BI dashboards

- This is a classic ETL pipeline

Example: Classic AI pipeline



Another Scenario

Scenario: Real-time fraud detection for a payments company

- Ingest / Extract
  - Stream transactions from POS terminals and mobile apps
  - Use Kafka/Event Hub
- Transform
  - Clean data
  - Generate features (velocity, geolocation mismatch, merchant risk score)
  - Apply ML model to score fraud probability in real time
- Load
  - Store scored transactions in a warehouse
  - Send high-risk alerts to fraud team dashboar

Types of Data Pipelines

A. **Batch processing:**
   a. Used for very large volumes of data moving in/out of warehouses
   b. Does not deal with real time data
   c. Usually scheduled.

B. **Stream processing**:
   a. Used for real time data
   b. Gives near zero time latency
   c. Used in real time analysis in time critical applications

C. **Cloud-native processing**
   a. Services are native to cloud infrastructure and cloud based data
   b. Tools are hosted on the cloud
   c. Hybrid mode- real time; quasi real time or batch
   d. Tools provided and maintained by cloud provider

Book Example

- Design a data pipeline to get data from data.gov.in for current daily prices of various commodity items from various mandis and store it in a CSV file.
- Assume the API key is available
- Need to know these:
  - Where is the data source
  - What type of data should be extracted
  - What transformations should be applied
  - What is the destination for data
  - Is the pipeline batch processing or stream ?
  - Who can trigger pipeline? Or is it Scheduled?

Book Example

- **Data Source Identification**: The data originates from the AGMARKNET portal, accessible via data.gov.in. Look for APIs or direct download links for daily commodity prices.
- **Data Extraction (Ingestion)**:
  - **Tool**: Python with requests library.
  - **Action**: Call the specific API endpoint for daily mandi prices, potentially filtering by date or commodity parameters if the API supports it.
- **Data Transformation**:
  - **Tool**: Python with pandas library.
  - **Action**:
    - Parse the JSON/XML response into a pandas DataFrame.
    - Clean data (handle missing values, correct formats).
    - Filter for specific commodities (e.g., 'Wheat', 'Rice') and mandis (e.g., 'Delhi', 'Mumbai').
    - Select relevant columns (Date, Commodity, Mandi, Min Price, Max Price, Modal Price).
- **Data Loading (Storage)**:
  - **Tool**: pandas.to_csv() function.
  - **Action**: Save the processed DataFrame to a CSV file (e.g., mandi_prices_YYYYMMDD.csv), ensuring daily or appending records with timestamps.
- **Scheduling & Orchestration**:
  - **Tools**:
    - **Simple**: Cron (Linux/macOS) or Task Scheduler (Windows) for basic daily runs.
    - **Advanced**: Apache Airflow for complex workflows, monitoring, and retries.
  - **Action**: Schedule the Python script to run daily after market closes.

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Topic 2: Introduction to ETL and ELT

ETL & ELT

Extract → Transform → Load

- Extract data from source systems
- Transform it in a staging/ETL engine
- Load the transformed data into the warehouse

Transforms data before loading

Used when:

- Transformations are heavy
- Data quality must be enforced before loading

Pros

Strong data quality control
Centralized transformation logic

Cons

Slower for large volumes
ETL engine becomes a bottleneck

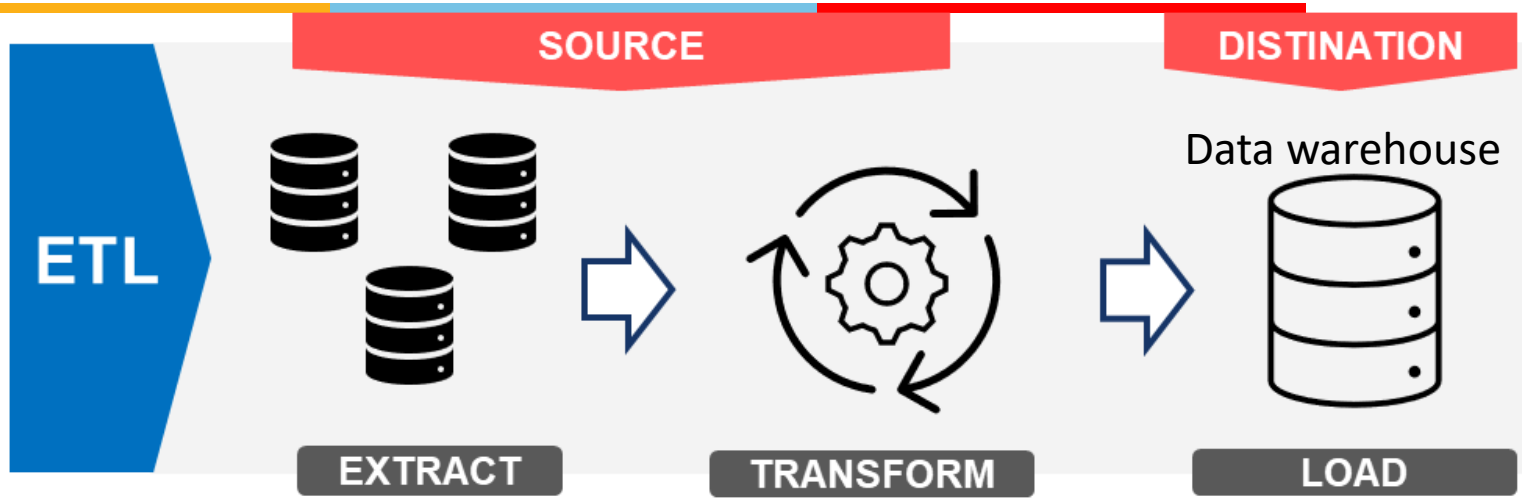
ELT

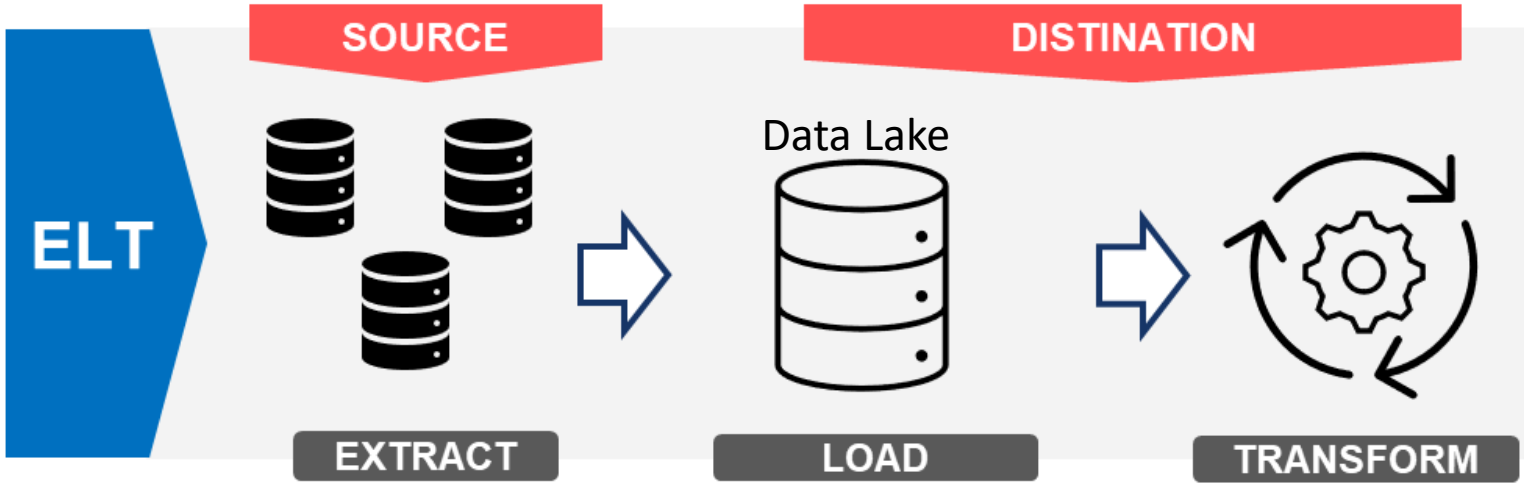
- For high velocity data and serving applications, transformation stage may be a bottleneck.
- Defer transformations to a time when data is needed by the application.
- Store data in a the raw form till then, in a Data Lake
- Process is now Extract – Load - Transform

ELT process

- **Process**: Data is extracted, loaded, and then transformed, shifting the transformation step from a staging server directly into the target data warehouse.
- **Advantages**: Faster loading times, enhanced scalability for big data, and flexibility to transform data on-demand.
- **Use Cases**: Highly suited for modern data lakes, real-time data ingestion, and cloud-based infrastructures.
- ELT vs. ETL: Unlike ETL (Extract, Transform, Load), which cleanses data before loading, ELT stores raw data, allowing for more flexible, on-demand analysis

ETL vs ELT


SOURCE
DISTINATION
ETL
Data warehouse
EXTRACT
TRANSFORM
LOAD


SOURCE
DISTINATION
ELT
Data Lake
EXTRACT
LOAD
TRANSFORM

ETV vs ELT comparison

aspect	ETL	ELT
Processing Order	Extract → Transform → Load	Extract → Load → Transform
Data transformation location	Transform before loading into the data warehouse	Transformation within the data warehouse
Immediate data	Data is not current because it is loaded after conversion	Data is highly current because it is converted after loading
Data Quality	Pre-conversion ensures high data quality	Data governance is required for later conversion
Scalability	There are limitations to processing large amounts of data	Suitable for processing large amounts of data
flexibility	Predefined transformations and limited flexibility	Conversions are done on demand for greater flexibility
cost	Can be costly for complex ETL processes and large datasets	Potentially cost-effective, especially for cloud-based solutions
Application Examples	Suitable for situations where high data quality and predefined analytical needs are required	Suitable for large amounts of data, real-time analysis, and agile data exploration
Applicable environment	On-premise, traditional DWH	Cloud DWH, Data Lake
Applying the Schema	Schema-on-write: define a schema before writing data to ensure consistency	Schema-on-read: not fixing the schema when saving data, but converting it into the required format when reading it
Data analysis	Use pre-organized data	Flexible analysis possible
Purpose	BI reports, batch processing	Machine learning, real-time analytics

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Topic 3: Data Lakes & Data Warehouse

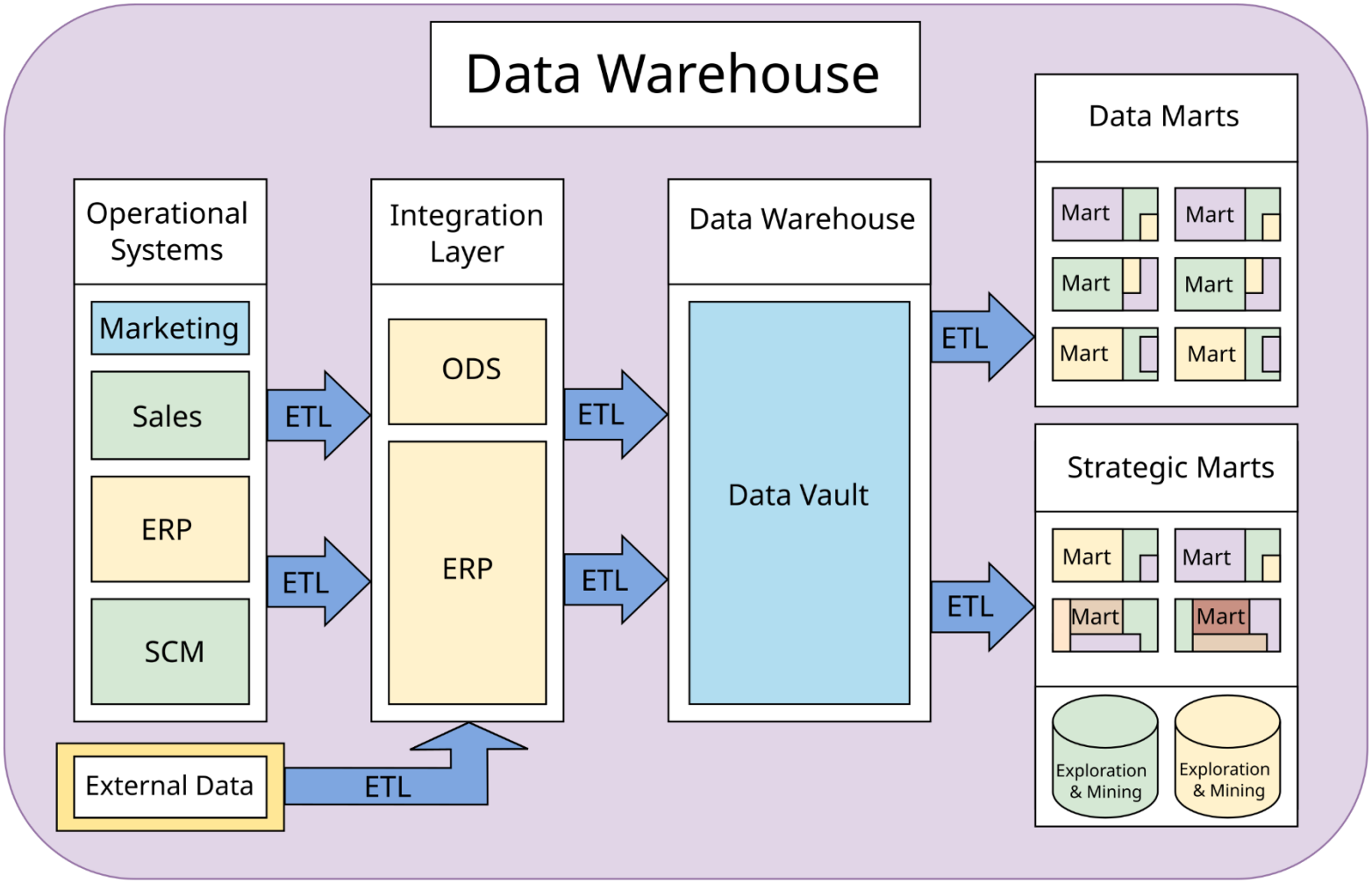
Data Warehousing

- A data warehouse is a centralized repository that aggregates structured current and historical data from multiple, disparate sources, designed specifically to support business intelligence (BI), analytics, and decision-making requirements.
- It serves as an organization's "single source of truth," allowing for fast, complex queries and reporting on large datasets.
- Essentially it is a Write-Once Read-Many type of a data store. No updates are performed. Data is only loaded from time to time.

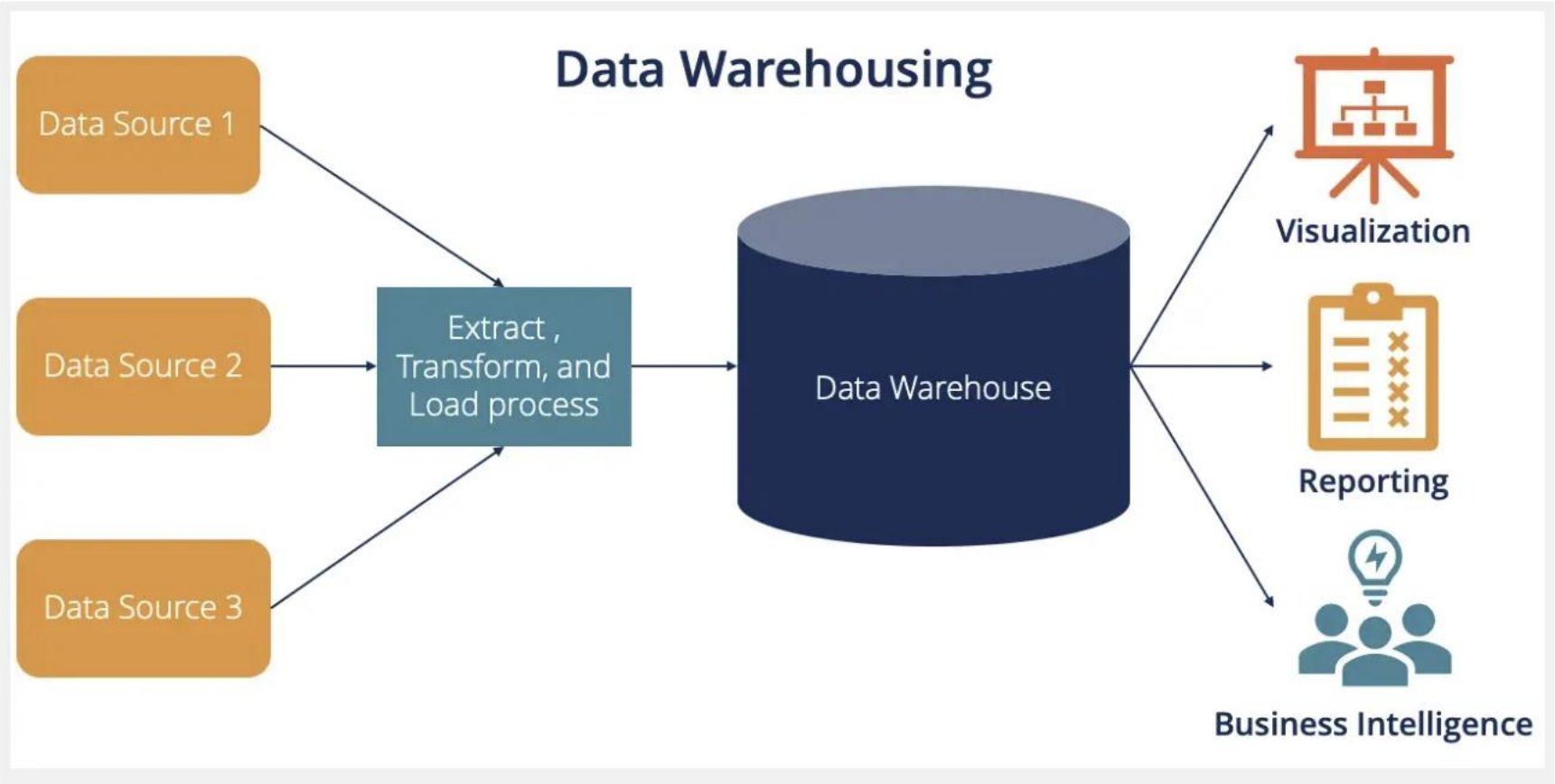
Data Warehouse

- Integrate data from multiple source systems, enabling a central view across the enterprise.
- Present the organization's information consistently.
- Provide a single common data model for all data of interest, regardless of data source.

The Datawarehouse



Data Warehousing



Schema in a Data Warehouse

Facts: Numeric transactional data

Dimensions: reference information that provides context to the data

Practical Scenario

- Data is not available in a single table.
- Data is spread out in multiple interconnected (interrelated) tables
  - To make effective use of storage
  - To remove duplicate occurrences
  - To maintain correctness and .. of data
  - To make retrieval easier and fast
- Enterprise databases / data sources use such storage structures

Employee Data

ADARSH TRADERS

Employee Id	Employee Name	Department Name	Location	Office Address	Mobile	Grade
101	EMP1	SALES	MUMBAI	kailash Park, Vikhroli , Mumbai , 400 001	9821234	JUNIOR
102	EMP2	SALES	MUMBAI	kailash Park, Vikhroli , Mumbai , 400 001	9822345	MID LEVEL
103	EMP3	ACCOUNTS	PUNE	Diamond Nagar, Near Akurdi, Pune, 420005	9813434	SENIOR
104	EMP4	HR	PUNE	Diamond Nagar, Near Akurdi, Pune, 420005	9840101	MID LEVEL
105	EMP5	HR	MUMBAI	kailash Park, Vikhroli , Mumbai , 400 001	9903214	SENIOR
106	EMP6	ADMIN	MUMBAI	kailash Park, Vikhroli , Mumbai , 400 001	9014567	JUNIOR
107	EMP7	ADMIN	PUNE	Diamond Nagar, Near Akurdi, Pune, 420005	9842121	SENIOR
108	EMP8	ACCOUNTS	PUNE	Diamond Nagar, Near Akurdi, Pune, 420005	9781212	MID LEVEL
109	EMP9	SALES	PUNE	Diamond Nagar, Near Akurdi, Pune, 420005	9772367	SENIOR
110	EMP10	ADMIN	MUMBAI	kailash Park, Vikhroli , Mumbai , 400 001	9890124	JUNIOR
111	EMP11	HR	MUMBAI	kailash Park, Vikhroli , Mumbai , 400 001	9678102	JUNIOR
112	EMP12	SALES	MUMBAI	kailash Park, Vikhroli , Mumbai , 400 001	9825697	MID LEVEL
113	EMP13	SALES	PUNE	Diamond Nagar, Near Akurdi, Pune, 420005	9837630	MIDLEVEL
114	EMP14	HR	PUNE	Diamond Nagar, Near Akurdi, Pune, 420005	9543403	SENIOR
115	EMP15	SALES	MUMBAI	kailash Park, Vikhroli , Mumbai , 400 001	9620205	JUNIOR
118	EMP18	SALES	MUMBAI	kailash Park, Vikhroli , Mumbai , 400 001	9780442	SENIOR
120	EMP20	SALES	PUNE	Diamond Nagar, Near Akurdi, Pune, 420005	9567290	JUNIOR
121	EMP21	SALES	PUNE	Diamond Nagar, Near Akurdi, Pune, 420005	9770145	SENIOR
124	EMP24	ADMIN	PUNE	Diamond Nagar, Near Akurdi, Pune, 420005	9895430	SENIOR
125	EMP25	SUPPORT	MUMBAI	kailash Park, Vikhroli , Mumbai , 400 001	9867822	MID LEVEL

Employee Data Restructured

Employee

Employee	Employee	D-Code	Address_Code	Mobile	Grade_Id
101	EMP1	D3	1	9821234	1
102	EMP2	D3	1	9822345	2
103	EMP3	D2	2	9813434	3
104	EMP4	D4	2	9840101	2
105	EMP5	D4	1	9903214	3
106	EMP6	D1	1	9014567	1
107	EMP7	D1	2	9842121	3
108	EMP8	D2	2	9781212	2
109	EMP9	D3	2	9772367	3
110	EMP10	D1	1	9890124	1
111	EMP11	D4	1	9678102	1
112	EMP12	D3	1	9825697	2
113	EMP13	D3	2	9837630	2
114	EMP14	D4	2	9543403	3
115	EMP15	D3	1	9620205	1
118	EMP18	D3	1	9780442	3
120	EMP20	D3	2	9567290	1
121	EMP21	D3	2	9770145	3
124	EMP24	D1	2	9895430	3
125	EMP25	D5	1	9867822	2

Department

Dept_id	Department Name
D1	ADMIN
D2	ACCOUNTS
D3	SALES
D4	HR
D5	SUPPORT

Grade

Gr_Id	Grade
1	JUNIOR
2	MID LEVEL
3	SENIOR

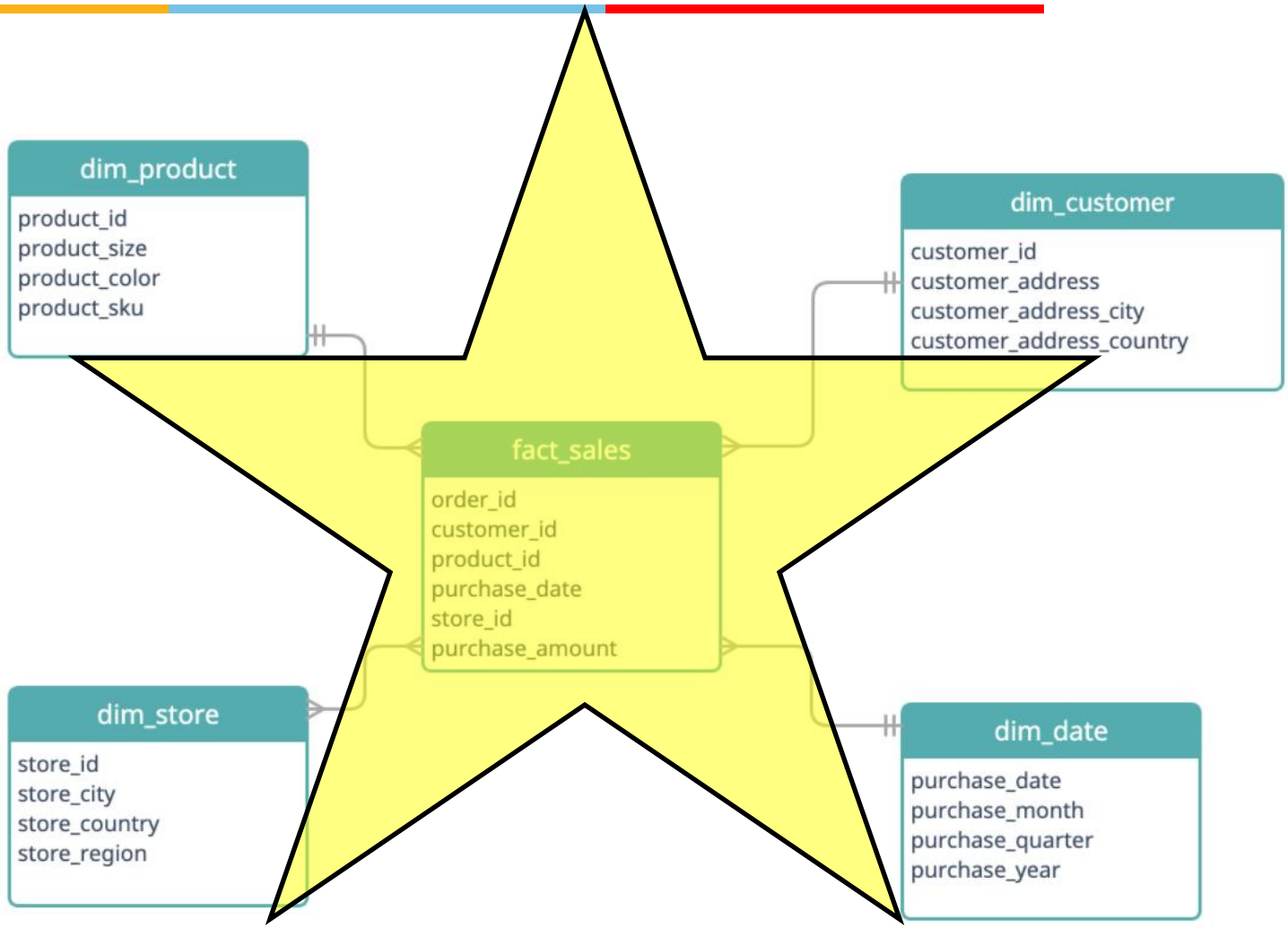
Office_Address

Address_Id	Postal Address	Location
1	kailash Park, Vikhroli , Mumbai , 400 001	MUMBAI
2	Diamond Nagar, Near Akurdi, Pune, 420005	PUNE

The Star Schema

- It organizes data into a central "**fact**" table and multiple connected "**dimension**" tables, visually resembling a star.
- In a star schema, the relationships between the fact table and dimension tables are typically many-to-one.
  - This means that many rows in the fact table can relate to a single row in a dimension table (e.g., many sales transactions can relate to one specific product).

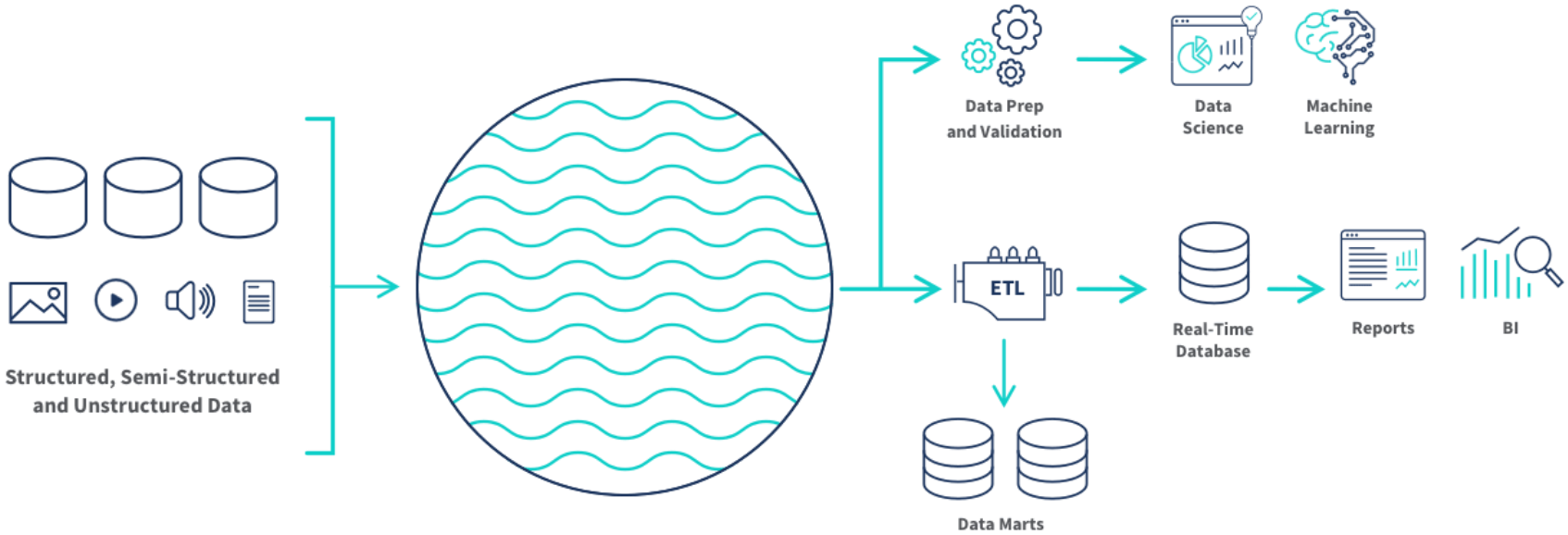
Th Star Schema



Data Lake

- A data lake is a data storage strategy whereby a centralized repository holds all of the organization's structured and unstructured data.
- It employs a flat architecture which permits to stores raw data at any scale without the need to structure it first.
- Instead of pre-defining the schema and data requirements, one uses tools to assign unique identifiers and tags to data elements so that only a subset of relevant data is queried to analyze a given business question
- Stores both structured and unstructured form of data eliminating need for separate types of stores

Data Lake Architecture



Downside of Data lakes

- Data lakes can become disorganized, making it hard to connect them with business intelligence and analytics tools.
- Lack of consistent data structure and ACID (atomicity, consistency, isolation, and durability) transactional support can result in sub-optimal query performance when required for reporting and analytics use cases.
- Providing data security and governance policies are difficult

Data lakehouse

- A data lakehouse is a new, big-data storage architecture that combines the best features of both data warehouses and data lakes.
- A data lakehouse enables a single repository for all the data (structured, semi-structured, and unstructured) while enabling best-in-class machine learning, business intelligence, and streaming capabilities.

Data lakehouse

- Data lakehouses usually start as data lakes containing all data types; the data is then converted to Delta Lake format (an open-source storage layer that brings reliability to data lakes).
- Delta lakes enable ACID transactional processes from traditional data warehouses on data lakes.

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Topic 4: Batch Jobs

The Data Pipeline Process

- Data Pipeline defines an end to end process.
- Process consists of a chain of tasks.
- Tasks may be made of small activities.

- How to trigger the pipeline?
  - Execute the script / program whenever needed. Manual launch.
  - Use some automation to trigger execution of the script periodically (say every 60 minutes)
- Scheduler helps us in scheduling the tasks to be triggered as per a ‘schedule’

The Scheduler

- A **scheduler** is the component that **automates, triggers, and orchestrates** the execution of data pipeline tasks.
- It is the **conductor of the pipeline orchestra**—deciding *when*, *how*, and *in what order* each step runs.
- Scheduler treats the execution context as a 'job' and it manages the job execution
  - Job has many tasks

Why schedulers?

Schedulers ensure that pipelines:

- Run **automatically** (no manual intervention)
- Run in the **correct sequence**
- Handle **dependencies** (e.g., “load only after extract finishes”)
- Recover from failures
- Scale across multiple jobs and environments

They are essential for ETL, ELT, and ITL workflows

What does a scheduler do?

- Triggers pipelines on:
  - Time (cron, hourly, daily, monthly)
  - Events (file arrival, API call, message in queue)
  - Conditions (upstream job success)
- Manages dependencies
- Retries failed tasks
- Sends alerts/notifications
- Logs execution history

Types of Scheduling

- Time-Based Scheduling
  - “Run every day at 2 AM”
  - Common in ETL batch jobs
- Event-Based Scheduling
  - “Start when a file lands in S3”
  - Common in ELT and ingestion pipelines
- Real-Time / Streaming Scheduling
  - Continuous execution
  - Common in ITL pipelines with ML scoring

Popular Scheduling Tools

- Enterprise / Cloud
  - Azure Data Factory
  - AWS Step Functions
  - Google Cloud Composer
  - Microsoft Fabric Pipelines
- Open-Source
  - Apache Airflow
  - Apache NiFi
  - Luigi
  - Prefect
- Legacy ETL Tools
  - Informatica Scheduler
  - SAP BODS job server
  - SSIS SQL Agent

Example Scheduler in Action

Scenario: Daily Sales ETL

2:00 AM → Extract POS data

2:15 AM → Transform & clean

2:30 AM → Load into warehouse

2:45 AM → Refresh Power BI dataset

Scheduler ensures:

Order

Timing

Error handling

Cron jobs

- cron is a software utility that allows us to schedule tasks on Unix –like systems.
- It is a time-based job scheduler in Unix-like systems that runs tasks automatically at specified intervals.
- It uses simple expressions (cron expressions) to define schedules like hourly, daily, or weekly job
- Tasks for cron are defined in a table called crontab

Example

Example:

0 2 * * *

This cron expression runs a job every day at 2:00 AM — perfect for daily ETL loads

Command:

0 2 * * * /usr/local/bin/run_etl.sh

Positions:
min (0-59) / hour (0-23) / day of month(1-31) / month (1-12) / day of week (0-6)

Python example

Command:

0 3 * * * /usr/bin/python3 /home/user/scripts/daily_report.py

Meaning:

0 3 * * * → run every day at 3:00 AM

/usr/bin/python3 → Python interpreter

/home/user/scripts/daily_report.py → the Python script

crontab

- A **crontab** is simply the **table of cron jobs** — the file where all the scheduled tasks are listed.
- In practical terms:
- **crontab = the configuration file that stores the cron schedules + commands**
- Each line in the crontab is one scheduled job (e.g., a Python script, ETL script, backup job)

example

```
# Run Python ETL script every day at 3:00 AM
0 3 * * * /usr/bin/python3 /home/user/scripts/daily_etl.py

# Run a log cleanup job every Sunday at midnight
0 0 * * 0 /home/user/scripts/cleanup_logs.sh

# Run a data backup every 6 hours
0 */6 * * * /usr/bin/python3 /home/user/scripts/backup.py
```

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Topic 5: Airflow / Cloud Overview

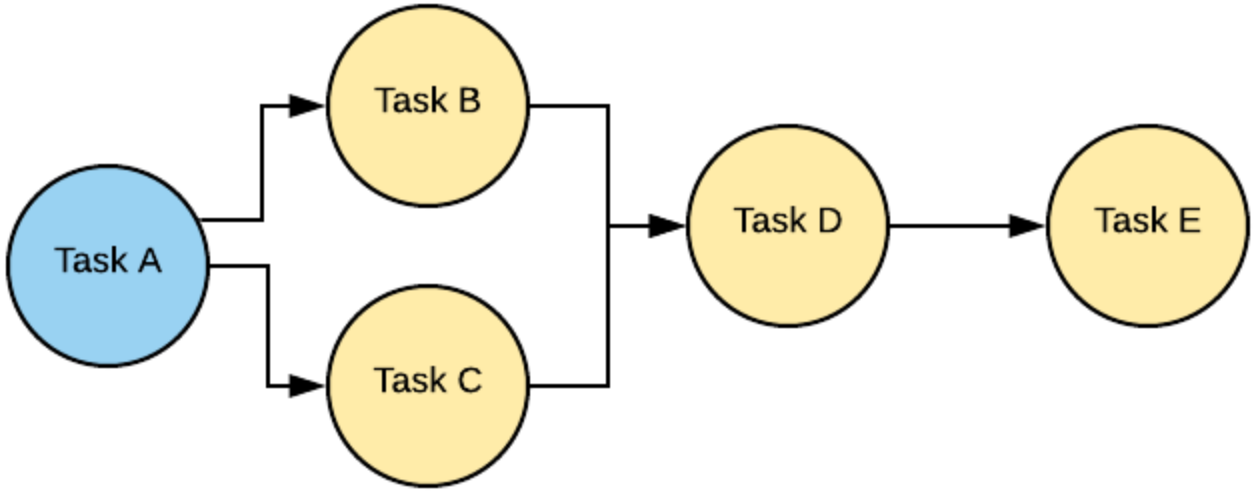
Apache Airflow

- A Platform to Orchestrate Data Pipelines
- Open-source workflow orchestration tool created by Airbnb
- Lets us define, schedule, and monitor pipelines as DAGs (Directed Acyclic Graphs)
- Ideal for ETL/ELT workflows, ML pipelines, and complex multi-step jobs
- Uses Python to define tasks, dependencies, and execution logic
- Key Idea:
  Airflow doesn't move data — it orchestrates the systems that do

Airflow DAG

- In Apache Airflow, a **DAG (Directed Acyclic Graph)** is a collection of tasks organized with dependencies and relationships, representing a workflow or a data pipeline.
- The DAG itself defines the structure and execution order of the tasks, not the actual data processing that happens within them

DAG in Airflow



Apache Airflow

Core Strengths:

- Python-based: Pipelines defined as code (flexible, version-controlled)
- Scheduler + Executor: Handles dependencies, retries, and parallelism
- UI Dashboard: Visualize DAGs, monitor runs, debug failures
- Integrations: Works with Spark, Snowflake, BigQuery, Databricks, AWS, Azure, GCP
- Extensible: Custom operators, sensors, hooks

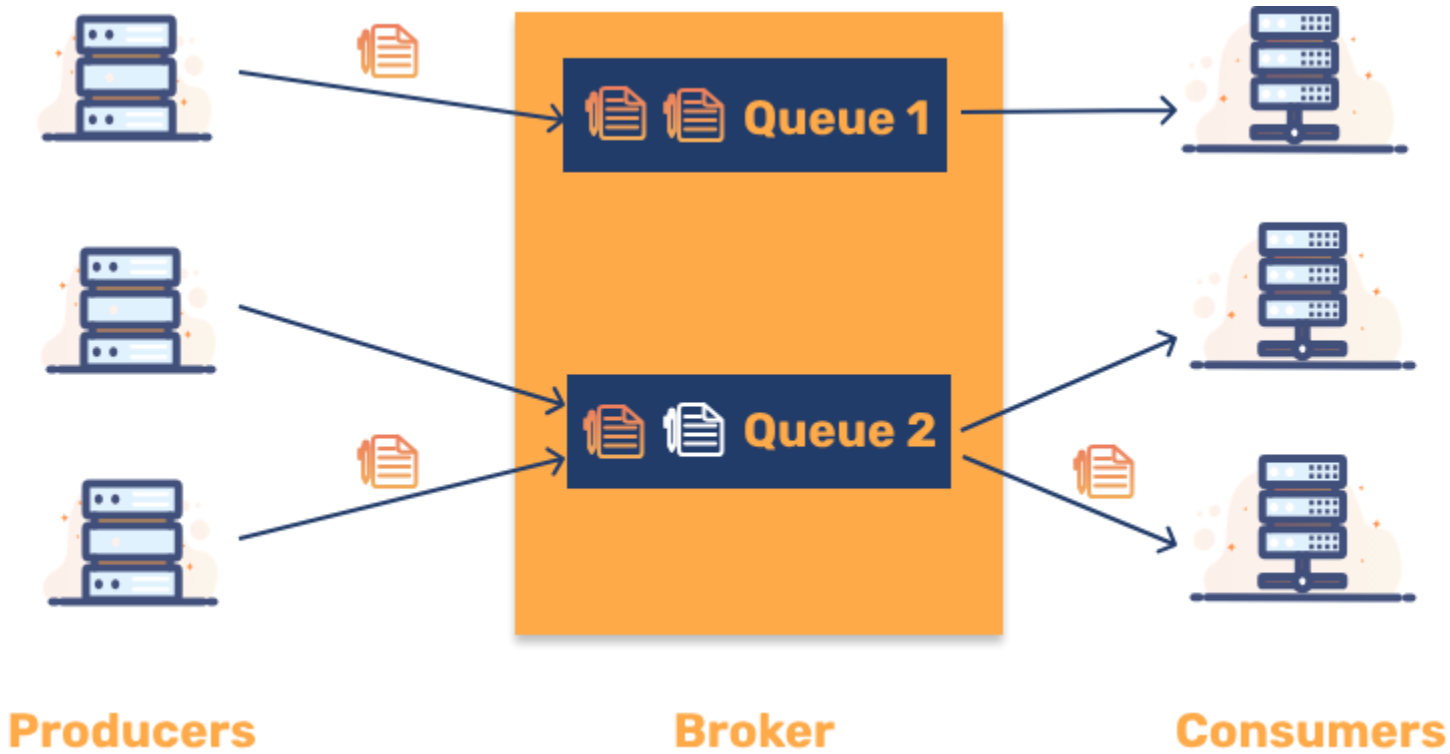
Best For:

- Batch ETL/ELT
- ML training + scoring workflows
- Data quality checks
- Multi-system orchestration

Message Communication

- Communication across tasks and services is realized using a Message Queue.
- Message Q sets up a Q that connects producers (of messages) and consumers (of messages).
- Whenever a message arrives from a producer, all (or a select few) subscribers are notified to act.
- Asynchronous architecture
- Producers send Messages to separate queues based on defined topics; consumers subscribe to topics to receive specific messages.
- **mqtt** is the protocol used for message exchange
- RabbitMQ is a popular MQ solution

Message Q



Summary

- Studies the concept of Data flows and data pipelines.
- Studied ETL and ELT and understood the differences
- Discussed what are Data warehouses and data lakes.
- Learnt the significance of a scheduler and understood the working of cron scheduler
- Studied Apache Airflow and understood the concept of message queues.

Questions?